

Splay

Splay 树

本页面将简要介绍如何用 Splay 维护二叉查找树。

定义

Splay 树，或 **伸展树**，是一种平衡二叉查找树，它通过 **伸展（splay）操作** 不断将某个节点旋转到根节点，使得整棵树仍然满足二叉查找树的性质，能够在均摊 时间内完成插入、查找和删除操作，并且保持平衡而不至于退化为链。

Splay 树由 Daniel Sleator 和 Robert Tarjan 于 1985 年发明。

基本结构与操作

本节讨论 Splay 树的基本结构和它的核心操作，其中最为重要的是伸展操作。

Splay 树是一棵二叉查找树，查找某个值时满足性质：左子树任意节点的值 < 根节点的值 < 右子树任意节点的值。

维护信息

本文使用数组模拟指针来实现 Splay 树，需要维护如下信息：

rt	id	fa[i]	ch[i][0/1]	val[i]	cnt[i]	sz[i]
----	----	-------	------------	--------	--------	-------

根节点编号 已使用节点个数 父亲 左右儿子编号 节点权值 权值出现次数 子树大小

初始化时，所有信息都置零即可。

辅助操作

首先是一些简单的辅助操作：

- `dir(x)`：判断节点 `x` 是父亲节点的左儿子还是右儿子；
- `push_up(x)`：在改变节点位置后，根据子节点信息更新节点 `x` 的信息。

▼ 实现

```
1 bool dir(int x) { return x == ch[fa[x]][1]; }
2
3 void push_up(int x) { sz[x] = cnt[x] + sz[ch[x][0]] + sz[ch[x][1]]; }
```

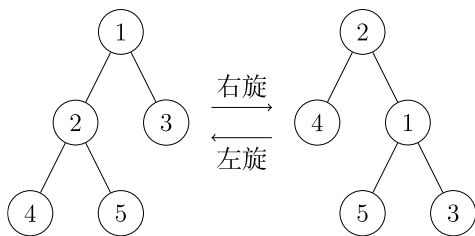
旋转操作

为了使 Splay 保持平衡，需要进行旋转操作。旋转的作用是将某个节点上移一个位置。

旋转需要保证：

- 整棵 Splay 的中序遍历不变（不能破坏二叉查找树的性质）；
- 受影响的节点维护的信息依然正确有效；
- `rt` 必须指向旋转后的根节点。

在 Splay 中旋转分为两种：左旋和右旋。



观察图示可知，如果要通过旋转将节点（左旋时的 和右旋时的 ）上移，则旋转的方向由该节点是其父节点的左节点还是右节点唯一确定。因此，实现旋转操作时，只需要将要上移的节点 传入即可。

具体分析旋转步骤：（假设需要上移的节点为 ，以右旋为例）

1. 首先，记录节点 的父节点 ，以及 的父节点 （可能为空），并记录 是 的左子节点还是右子节点；
2. 按照旋转后的树中自下向上的顺序，依次更新 的左子节点为 的右子节点， 的右子节点为 ，以及若 非空， 的子节点为 ；
3. 按照同样的顺序，依次更新当前 的左子节点（若存在）的父节点为 ， 的父节点为 ，以及 的父节点为 ；
4. 自下而上维护节点信息。

▼ 实现

```
1 void rotate(int x) {
2     int y = fa[x], z = fa[y];
3     bool r = dir(x);
4     ch[y][r] = ch[x][!r];
5     ch[x][!r] = y;
6     if (z) ch[z][dir(y)] = x;
7     if (ch[y][r]) fa[ch[y][r]] = y;
8     fa[y] = x;
9     fa[x] = z;
10    push_up(y);
11    push_up(x);
12 }
```

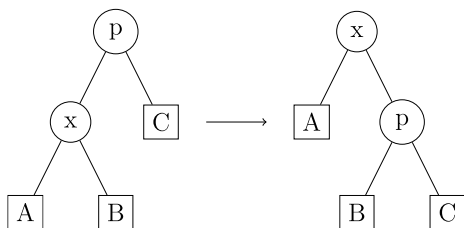
在所有函数的实现时，都应注意不要修改节点 的信息。

伸展操作

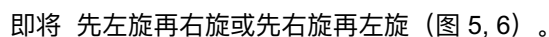
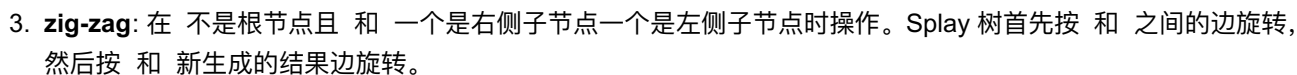
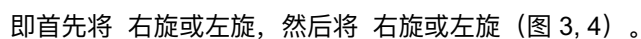
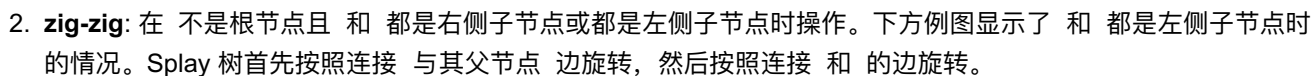
Splay 树要求每访问一个节点 后都要强制将其旋转到根节点。该操作也称为伸展操作。

设刚访问的节点为 。要做伸展操作，就是要对 做一系列的 **伸展步骤**。每次对 做一次伸展步骤， 到根节点的距离都会更近。定义 为 的父节点。伸展步骤有三种：

1. **zig**: 在 是根节点时操作。Splay 树会根据 和 间的边旋转。**zig** 存在是用于处理奇偶校验问题，仅当 在伸展操作开始时具有奇数深度时作为伸展操作的最后一步执行。



即直接将 右旋或左旋（图 1, 2）。



请读者尝试自行模拟 种旋转情况，以理解伸展操作的基本思想。

比较三种伸展步骤可知，要区分此时应使用哪种操作，关键是要判断 是否是根节点的子节点，以及 和它父节点是否

在各自的父节点同侧。

此处提供的实现，可以指定任意根节点，并将它的子树内任意节点 上移至 处：

1. 首先记录根节点的父节点，从而可以利用 $fa[x] == w$ 判断 已经位于根结点处；
2. 记录 当前的父节点，如果 和 相同，说明 已经到达根节点；
3. 否则，利用 $fa[y] == w$ 判断 是否是根节点。如果是，直接做 zig 操作将 旋转；如果不是，利用 $dir(x) == dir(y)$ 判断使用 zig-zig 还是 zig-zag，前者先旋转 再旋转，后者直接旋转两次。

▼ 实现

```
1 void splay(int& z, int x) {
2     int w = fa[z];
3     for (int y; (y = fa[x]) != w; rotate(x)) {
4         if (fa[y] != w) rotate(dir(x) == dir(y) ? y : x);
5     }
6     z = x;
7 }
```

伸展操作是 Splay 树的核心操作，也是它的时间复杂度能够得到保证的关键步骤。请务必保证每次向下访问节点后，都进行一次伸展操作。

另外，伸展操作会将当前节点 到根节点 的路径上的所有节点信息自下而上地更新一遍。正是因为这一点，才可以修改非根节点，再通过伸展操作将它上移至根来完成整个树的信息更新。

时间复杂度

对大小为 的 Splay 树做 次伸展操作的复杂度是 的，单次均摊复杂度是 的。

- 基于势能分析的复杂度证明
- 为什么 Splay 树的再平衡操作可以获得 的均摊复杂度？

平衡树操作

本节讨论基于 Splay 树实现平衡树的常见操作的方法。其中，较为重要的是按照值或排名查找元素，它们可以将某个特定的元素找到，并上移至根节点处，以便后续处理。

作为例子，本节将讨论模板题目 [普通平衡树](#) 的实现。

按照值查找

作为二叉查找树，可以通过值 查找到相应的节点，只需要将待查找的值 和当前节点的值比较即可，找到后将该元素上移至根部即可。

应注意，经常存在树中不存在相应的节点的情形。对于这种情形，要记录最后一个访问的节点（即实现中的 ），并将上移至根部。此时，节点 存储的值必然要么是所有小于 的元素中最大的（即 的前驱），要么是所有大于 的元素中最小的（即 的后继）。这是因为查找过程保证，左子树总是存储小于 的值，而右子树总是存储大于 的值。

▼ 实现

```
1 void find(int& z, int v) {
2     int x = z, y = fa[x];
3     for (; x && val[x] != v; x = ch[y = x][v > val[x]]);
4     splay(z, x ? x : y);
5 }
```

该实现允许指定任何节点 作为根节点，并在它的子树内按值查找。

按照排名访问

因为记录了子树大小信息，所以 Splay 树还可以通过排名访问元素，即查找树中第 k 小的元素。

设 z 为剩余排名，具体步骤如下：

- 如果左子树非空且剩余排名 z 不大于左子树的大小，那么向左子树查找；
- 否则，如果 z 不大于左子树加上根的大小，那么根节点就是要寻找的；
- 否则，将 z 减去左子树的和根的大小，继续向右子树查找；
- 将最终找到的元素上移至根部。

▼ 实现

```
1 void loc(int& z, int k) {
2     int x = z;
3     for (;;) {
4         if (sz[ch[x][0]] >= k) {
5             x = ch[x][0];
6         } else if (sz[ch[x][0]] + cnt[x] >= k) {
7             break;
8         } else {
9             k -= sz[ch[x][0]] + cnt[x];
10            x = ch[x][1];
11        }
12    }
13    splay(z, x);
14 }
```

该实现需要保证排名 k 不超过根 z 处的树大小。

模板题目中操作 `kth` 要求按照排名返回值，直接调用该方法，并返回值即可。

▼ 实现

```
1 int find_kth(int k) {
2     if (k > sz[rt]) return -1;
3     loc(rt, k);
4     return val[rt];
5 }
```

合并操作

有些时候需要合并两棵 Splay 树。

设两棵树的根节点分别为 x 和 y ，那么为了保证结果仍是二叉查找树，需要要求 x 树中的最大值小于 y 树中的最小值。这条条件通常都可以满足，因为两棵树往往是从更大的子树中分裂出的。

合并操作如下：

- 如果 x 和 y 其中之一或两者都为空树，直接返回不为空的那一棵树的根节点或空树；
- 否则，通过 `loc(y, 1)` 将 y 树中的最小值上移至根 y 处，再将它的左节点（此时必然为空）设置为 x ，并更新节点信息，返回节点 y 。

▼ 实现

```

1 int merge(int x, int y) {
2   if (!x || !y) return x | y;
3   loc(y, 1);
4   ch[y][0] = x;
5   fa[x] = y;
6   push_up(y);
7   return y;
8 }

```

分裂操作类似。因而，Splay 树可以模拟 [无旋 treap](#) 的思路做各种操作，包括区间操作。[后文](#) 会介绍更具有 Splay 树风格的区间操作处理方法。

插入操作

插入操作是一个比较复杂的过程。具体步骤如下：（假设插入的值为 v ）

- 类似按值查找的过程，根据 v 向下查找到存储 v 的节点或者空节点，过程中记录父节点；
- 如果存在存储 v 的节点，直接更新信息，否则就新建节点；
- 做伸展操作，将最后一个节点 x 上移至根部。

▼ 实现

```

1 void insert(int v) {
2   int x = rt, y = 0;
3   for (; x && val[x] != v; x = ch[y = x][v > val[x]]);
4   if (x) {
5     ++cnt[x];
6     ++sz[x];
7   } else {
8     x = ++id;
9     val[x] = v;
10    cnt[x] = sz[x] = 1;
11    fa[x] = y;
12    if (y) ch[y][v > val[y]] = x;
13  }
14  splay(rt, x);
15 }

```

该实现允许直接向空树内插入值。若不想处理空树，可以在树中提前插入哑节点。

删除操作

删除操作也是一个比较复杂的操作。具体步骤如下：（假设删除的值为 v ）

- 首先按照值 v 查找存储它的节点，并上移至根部；
- 如果不存在存储它的节点，直接返回；（上一步已经做了伸展操作）
- 否则，更新节点信息；
- 如果得到的根节点为空节点，就合并左右子树作为新的根节点，注意合并前需要更新两个子树的根的父节点为空。

▼ 实现

```

1 bool remove(int v) {
2     find(rt, v);
3     if (!rt || val[rt] != v) return false;
4     --cnt[rt];
5     --sz[rt];
6     if (!cnt[rt]) {
7         int x = ch[rt][0];
8         int y = ch[rt][1];
9         fa[x] = fa[y] = 0;
10        rt = merge(x, y);
11    }
12    return true;
13 }

```

查询排名

直接按照值 访问节点（并上移至根），然后返回相应的值即可。

注意，当 不存在时，方法 `find(rt, v)` 返回的根和 的大小关系无法确定，需要单独讨论。

▼ 实现

```

1 int find_rank(int v) {
2     find(rt, v);
3     return sz[ch[rt][0]] + (val[rt] < v ? cnt[rt] : 0) + 1;
4 }

```

查询前驱

前驱定义为小于 的最大的数。具体步骤如下：

- 按照值 访问节点（并上移至根部）；
- 如果根部的值小于 ，那么它必然是最大的那个，直接返回；
- 否则，在左子树中找到最大值，并上移至根部。

最后一步相当于直接调用 `loc(ch[rt][0], cnt[ch[rt][0]])`，只是省去了不必要的判断。

▼ 实现

```

1 int find_prev(int v) {
2     find(rt, v);
3     if (rt && val[rt] < v) return val[rt];
4     int x = ch[rt][0];
5     if (!x) return -1;
6     for (; ch[x][1]; x = ch[x][1]);
7     splay(rt, x);
8     return val[rt];
9 }

```

该实现允许前驱不存在，此时返回 。

查询后继

后继定义为大于 的最小的数。查询方法和前驱类似，只是将左子树的最大值换成了右子树的最小值，即调用

`loc(ch[rt][1], 1)`。

▼ 实现

```

1 int find_next(int v) {
2     find(rt, v);
3     if (rt && val[rt] > v) return val[rt];
4     int x = ch[rt][1];
5     if (!x) return -1;
6     for (; ch[x][0]; x = ch[x][0]);
7     splay(rt, x);
8     return val[rt];
9 }

```

参考实现

本节的最后，给出模板题目 [普通平衡树](#) 的参考实现。

► 参考实现

序列操作

Splay 树也可以运用在序列上，用于维护区间信息。与线段树对比，Splay 树常数较大，但是支持更复杂的序列操作，如区间翻转等。上文提到 Splay 树同样支持分裂和合并操作，因而可以模拟 [无旋 treap](#) 进行区间操作，在此不再过多讨论。本节主要讨论基于伸展操作的区间操作实现方法。

将序列建成的 Splay 树有如下性质：

- Splay 树的中序遍历相当于原序列从左到右的遍历；
- Splay 树上的一个节点代表原序列的一个元素；
- Splay 树上的一颗子树，代表原序列的一段区间。

因为有伸展操作，可以快速提取出代表某个区间的 Splay 子树。

作为例子，本节将讨论模板题目 [文艺平衡树](#) 的实现。

根据序列建树

在操作之前，需要根据所给的序列先把 Splay 树建出来。根据 Splay 树的特性，直接建出一颗只有左儿子的链即可。时间复杂度是 $O(n^2)$ 的。

▼ 参考实现

```

1 void build(int n) {
2     for (int i = 1; i <= n + 2; ++i) {
3         ++id;
4         ch[id][0] = rt;
5         if (rt) fa[rt] = id;
6         rt = id;
7         val[id] = i - 1;
8     }
9     splay(rt, 1);
10 }

```

最后的伸展操作自下而上地更新了节点信息。为了后文区间操作方便，序列左右两侧添加了两个哨兵节点。

区间翻转

以区间翻转为例，可以理解区间操作的方法：（设区间为 $[l, r]$ ）

- 首先将节点 l 上移到根节点，再在其右子树中，将节点 r 上移到右子树的根节点；
- 此时，设 l 为根节点的右子节点的左子节点，则以 r 为根的子树就对应着区间 $[l, r]$ ；

- 在 `loc` 处对区间 做操作，并打上懒标记；
- 在 `loc` 处将标记下传一次，然后利用伸展操作将 `loc` 上移到根。

第一步需要的操作就是前文平衡树操作中的「按照排名访问」，因为元素的标号就是它的排名。因为涉及懒标记的管理，它的实现与上文略有不同。

▼ 参考实现

```
1 void reverse(int l, int r) {
2     loc(rt, l);
3     loc(ch[rt][1], r - l + 2);
4     int x = ch[ch[rt][1]][0];
5     lazy_reverse(x);
6     push_down(x);
7     splay(rt, x);
8 }
```

最后一步的伸展操作并非为了保证复杂度正确，而是为了更新节点信息。因为伸展操作涉及到节点 `loc` 的左右子节点，所以之前需要将节点 `loc` 处的标记先下传一次。当然，仅对于区间翻转操作而言，子区间的翻转不会对祖先节点产生影响，所以省去这一步骤也是正确的。此处实现保留这两行，是为了说明一般的情形下的操作方法。

懒标记管理

首先，需要辅助函数 `lazy_reverse(x)` 和 `push_down(x)`。前者交换左右节点，并更新懒标记；后者将标记下传。

▼ 参考实现

```
1 void lazy_reverse(int x) {
2     std::swap(ch[x][0], ch[x][1]);
3     lz[x] ^= 1;
4 }
5
6 void push_down(int x) {
7     if (lz[x]) {
8         if (ch[x][0]) lazy_reverse(ch[x][0]);
9         if (ch[x][1]) lazy_reverse(ch[x][1]);
10        lz[x] = 0;
11    }
12 }
```

然后，只需要在向下经过节点时下传标记即可。模板题要求的操作比较简单，只有按照排名寻找的操作（即 `loc`）涉及向下访问节点。注意，需要在函数每次访问一个新的节点 前 下传标记。

▼ 参考实现

```
1 void loc(int& z, int k) {
2     int x = z;
3     for (push_down(x); sz[ch[x][0]] != k - 1; push_down(x)) {
4         if (sz[ch[x][0]] >= k) {
5             x = ch[x][0];
6         } else {
7             k -= sz[ch[x][0]] + 1;
8             x = ch[x][1];
9         }
10    }
11    splay(z, x);
12 }
```

因为向下访问节点时已经移除了经过的路径的所有懒标记，所以利用伸展操作上移节点时不再需要处理懒标记。但是，对于区间操作的那一个节点要谨慎处理：因为它同样位于伸展操作的路径上，但是刚刚操作完，可能存在尚未下传的标

记，需要首先下传再做伸展操作，正如同上文所做的那样。

参考实现

本节的最后，给出模板题目 [文艺平衡树](#) 的参考实现。

► 参考实现

习题

这些题目都是裸的 Splay 树维护二叉查找树：

- [【模板】普通平衡树](#)
- [【模板】文艺平衡树](#)
- [「HNOI2002」营业额统计](#)
- [「HNOI2004」宠物收养所](#)

Splay 树还出现在更复杂的应用场景中：

- [「Cerc2007」robotic sort 机械排序](#)
- [「HNOI2011」括号修复/「JSOI2011」括号序列](#)
- [二逼平衡树（树套树）](#)
- [BZOJ 2827 千山鸟飞绝](#)
- [「Lydsy1706 月赛」K 小值查询](#)
- [POJ3580 SuperMemo](#)

参考资料与注释

本文部分内容引用于 algocode 算法博客，特别鸣谢！

本页面最近更新：2025/3/9 18:33:30, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[NachtgeistW](#), [shuzhouliu](#), [StudyingFather](#), [abc1763613206](#), [Alpacabla](#), [aofall](#), [c-forrest](#), [Catreap](#), [CoelacanthusHex](#), [countercurrent-time](#), [Early0v0](#), [Enter-tainer](#), [ezoixx130](#), [GavinZhengOI](#), [Gesruea](#), [GrapeLemonade](#), [Great-designer](#), [H-J-Granger](#), [Henry-ZHR](#), [HeRaNO](#), [hly1204](#), [hsfzLZH1](#), [iamtwz](#), [lr1d](#), [isdanni](#), [ksyx](#), [longlongzhu123](#), [Macesuted](#), [Marcythm](#), [mcendu](#), [Molmin](#), [ouuan](#), [partychicken](#), [Persdre](#), [saigonoinorio](#), [Siyuan](#), [sshwy](#), [SukkaW](#), [Tiphereth-A](#), [Xeonacid](#), [xiaofu-15191](#), [yuhuoji](#), [yyyu-star](#), [zcz0263](#), [zzxLLLL](#), [代建杉](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用